

50 SDET Interview Questions

Very Simple Answers + Practical Examples

Based on the 50-question source PDF provided by the user.

How to use this guide

For every question, first try answering aloud in 60–90 seconds. Then compare your answer with the simple answer and example. Do not memorise the wording; understand the idea and adapt the example to your own project.

Interview structure used: Simple Answer → Practical Example → Interview Tip

Q1. What is the difference between smoke, sanity and regression testing, and where does each sit in a release cycle?

Simple Answer

Smoke testing is a quick check that the build is stable enough for deeper testing. Sanity testing is a focused check of a particular change or fresh build. Regression testing checks that existing functionality still works after changes.

Practical Example

Think of a banking application. After deployment, I first verify login, dashboard and a basic payment flow as smoke tests. If the payment feature was changed, I run focused payment-related sanity tests. Before release, I run the broader regression suite covering important existing journeys.

Interview Tip

Smoke = "Can I start testing?" Sanity = "Does this specific change look healthy?" Regression = "Did the change break something that already worked?"

Q2. How do you decide what NOT to automate?

Simple Answer

I do not automate everything. I consider execution frequency, stability, maintenance cost, business risk and whether the test is suitable for automation. One-time or highly exploratory tests may be better done manually.

Practical Example

If a UI page changes every week and a test is run only once a quarter, automating it may create more maintenance work than value. A frequently executed critical API regression test is a much stronger automation candidate.

Interview Tip

A good answer shows that automation is an engineering investment, not a goal by itself.

Q3. A test passes on your machine and fails in CI. Walk me through your debugging order.

Simple Answer

First I reproduce the failure using the same CI inputs. Then I compare environment, browser, dependency versions, test data, timing, parallel execution and network conditions. Finally I inspect screenshots, videos, logs and reports before changing the test.

Practical Example

For example, a test passes locally but fails in Jenkins because CI runs in UTC while the application test data expects IST. Comparing the environment and timestamps exposes the real problem instead of adding a random wait.

Interview Tip

Do not immediately change the code. First prove whether the difference is environment, data, timing or the test itself.

Q4. What makes a test flaky? Name causes you have personally fixed.

Simple Answer

A flaky test gives different results without a relevant product change. Common causes are hard waits, race conditions, shared data, order dependency, animations, unstable selectors, network timing and environment instability.

Practical Example

A checkout test sometimes failed because the payment button became enabled asynchronously. Replacing a fixed sleep with a condition-based wait made the test wait for the actual state instead of an arbitrary number of seconds.

Interview Tip

Mention two real examples from your experience. Interviewers are checking whether you have actually dealt with flakiness.

Q5. Your suite is green every day, yet bugs still escape to production. What is wrong?

Simple Answer

A green suite does not automatically mean good coverage. The tests may have shallow assertions, cover only stable paths, skip failures, use unrealistic data or run against an environment different from production.

Practical Example

Suppose login is tested successfully, but no test verifies that User A cannot see User B's data. The suite can remain 100% green while a serious authorization defect reaches production.

Interview Tip

I would review escaped defects and add tests for the failure patterns we are missing, rather than simply adding more test cases.

Q6. How do you measure the quality of a test suite?

Simple Answer

I look beyond test-count or code-coverage percentage. Useful measures include escaped defects, defect detection before merge, flaky-test rate, execution time, failure diagnosis time and how often the suite catches real regressions.

Practical Example

If a 2,000-test suite has 95% code coverage but never catches production defects, its percentage is not telling the whole story. A smaller reliable suite that repeatedly catches important regressions can provide more value.

Interview Tip

Coverage tells us what code was exercised; it does not prove that the behaviour was verified correctly.

Q7. You have two hours before release and four hundred tests. What do you run?

Simple Answer

I use risk-based testing. I prioritise what changed, critical customer journeys, high-risk integrations and areas with recent defects. I run the highest-value tests first rather than trying to run all 400 tests.

Practical Example

If a release changes claims processing, I prioritise login, eligibility, claim submission, payment-related APIs and the changed modules before unrelated UI tests.

Interview Tip

The key phrase is risk-based selection: change impact + business criticality + defect history.

Q8. Explain the Page Object Model. When does it break down?

Simple Answer

Page Object Model keeps page locators and page actions in page classes so tests describe business intent instead of implementation details.

Practical Example

A LoginPage can contain username, password and login-button locators and a login() method. The test simply calls loginPage.login(user, password). If the locator changes, I update the page rather than every test.

Interview Tip

POM can become difficult when it creates huge base classes, excessive abstraction or components that are shared across many pages. Modern applications often need reusable component objects rather than one giant page hierarchy.

Q9. Explicit versus implicit waits. And why is a hard sleep in a test considered a serious smell?

Simple Answer

An explicit wait waits for a specific condition, such as an element becoming visible or clickable. An implicit wait tells the driver to wait while locating elements. A hard sleep always waits a fixed amount of time.

Practical Example

If an API response normally takes 2 seconds but sometimes 5 seconds, sleep(5) wastes time on fast runs and may still fail when the response takes 6 seconds. Waiting for the actual condition is more reliable.

Interview Tip

A useful phrase: "Wait for truth, not time."

Q10. The element you need has no id and no stable attributes. What do you do?

Simple Answer

I first look for a stable accessibility attribute, role, text or relationship to a stable parent. I can use a relative locator when appropriate. If there is still no reliable selector, I ask developers to add a stable test identifier.

Practical Example

Instead of using a long XPath based on five nested divs, I would ask for data-testid='submit-payment'. That makes the test easier to maintain and less sensitive to UI layout changes.

Interview Tip

Do not turn locator archaeology into a permanent automation strategy.

Q11. How would you make a 4-hour suite finish in 15 minutes?

Simple Answer

I would first measure where the time is going. Then I would parallelise independent tests, shard the suite across workers, remove redundant UI tests, move suitable checks to API or unit level, and optimise setup and test data.

Practical Example

If 1,200 independent tests take 4 hours sequentially, I can divide them across multiple CI workers. If the effective work is evenly distributed, increasing parallelism can reduce wall-clock time dramatically.

Interview Tip

Mention both infrastructure parallelism and reducing unnecessary UI coverage.

Q12. What is the test pyramid, and where do most teams get it wrong?

Simple Answer

The test pyramid recommends many fast unit tests, fewer API/integration tests and a smaller number of UI end-to-end tests. UI tests are slower and generally harder to diagnose.

Practical Example

For a checkout feature, business rules can be tested at unit/API level, while only a few critical UI tests verify the complete customer journey.

Interview Tip

A common mistake is an inverted pyramid: hundreds of slow UI tests and very few fast lower-level tests.

Q13. How do you manage test data so that a thousand runs a day do not corrupt each other?

Simple Answer

Tests should create isolated data per run or worker, avoid shared mutable accounts, and clean up when possible. Data factories and unique identifiers are useful at scale.

Practical Example

Instead of every test using user@test.com, create user_<runId>_<workerId>. This prevents one parallel test from changing another test's account state.

Interview Tip

At scale, test data is part of the framework architecture, not a spreadsheet maintained manually.

Q14. How do you keep tests independent, and why does order dependence keep creeping in?

Simple Answer

Each test should create the state it needs and should not rely on another test running first. I also run tests in random order or parallel in CI to expose hidden dependencies.

Practical Example

Test B passes only because Test A created an account. When B runs alone it fails. The fix is for B to create its own account or use a controlled fixture.

Interview Tip

A test that passes only in one order is hiding shared state.

Q15. Selenium, Playwright, Cypress, Appium. How do you actually choose?

Simple Answer

I choose based on application type, browser/device needs, language skills, ecosystem, parallel execution, debugging, CI support and long-term maintenance—not personal preference.

Practical Example

For a web application needing strong cross-browser automation, I may choose Playwright or Selenium. For native mobile applications, Appium is relevant. The final decision depends on project constraints.

Interview Tip

Explain the decision criteria first and the tool second.

Q16. Describe a framework you built or significantly extended. What would you change today?

Simple Answer

I would explain the architecture, why the design was chosen, how tests run in CI, reporting, test data and the problems the framework solved. Then I would honestly mention one design decision I would improve today.

Practical Example

For example, I might describe a Selenium-to-Playwright migration with Page Objects, reusable browser/assertion utilities, selector management, CI integration and parallel execution. I could then explain where an earlier abstraction became too complex and how I would simplify it.

Interview Tip

The second part—what you would change—is testing your engineering maturity and self-awareness.

Q17. A known-flaky test sits in your merge gate. Retry it, quarantine it, or delete it? Defend your choice.

Simple Answer

I would not blindly retry it forever. If it is genuinely flaky, quarantine it temporarily with an owner, tracking ticket and deadline, while fixing the root cause. If the test has no value, delete it. A retry should not hide a real defect.

Practical Example

If a payment test intermittently fails because of a race condition, I quarantine it with an owner and investigate. If the same failure later proves to be a real product bug, I do not label it simply as flakiness.

Interview Tip

The goal is to preserve trust in the merge gate.

Q18. String manipulation live: reverse words in a sentence, in place.

Simple Answer

A common approach is to reverse the entire character sequence and then reverse each individual word. This changes the order of the words while preserving the characters inside each word.

Practical Example

For “I love Java”, reverse everything first and then reverse each word to obtain “Java love I”. In an interview, explain the algorithm while coding instead of silently typing.

Interview Tip

The interviewer is checking basic coding fluency and your ability to reason while coding.

Q19. Find duplicates in a large dataset. Follow-up: the data does not fit in memory.

Simple Answer

For data that fits in memory, I can use a HashSet to detect values seen before. If it does not fit, I can split the data into chunks, sort externally, or hash records into buckets and process each bucket separately.

Practical Example

For a 500 GB log file on a machine with 16 GB RAM, I cannot load everything into a HashSet. I can hash records into multiple files so each bucket is small enough to process independently.

Interview Tip

The follow-up tests whether you can scale the basic solution.

Q20. Parse a log file and return the top five most frequent error codes.

Simple Answer

I would parse each valid log line, count error codes in a map, and then use a min-heap of size five or sort the counts to find the top five.

Practical Example

If logs contain 500, 404, 500, 401, 500, then 500 has frequency 3. I would also handle malformed lines by skipping them or recording them separately rather than crashing the whole parser.

Interview Tip

Mention malformed input because production logs are rarely perfectly clean.

Q21. Write a retry helper with exponential backoff and a maximum attempt count.

Simple Answer

Retry means trying an operation again when the failure is temporary. Exponential backoff increases the delay after each failure, for example 1s, 2s, 4s, 8s. I would also add jitter and retry only known transient errors.

Practical Example

If a service returns HTTP 503 temporarily, retries may help. But I would not blindly retry an assertion failure because that can hide a real product defect.

Interview Tip

A senior answer includes retryable versus non-retryable failures and a maximum retry count.

Q22. Implement a simple rate limiter class.

Simple Answer

A rate limiter controls how frequently requests are allowed. A token-bucket design keeps a capacity of tokens and refills them at a fixed rate. Each request consumes a token.

Practical Example

If capacity is 10 and the refill rate is 2 tokens per second, a client can make an initial burst of up to 10 requests and then continues at roughly 2 requests per second as tokens refill.

Interview Tip

Token bucket handles bursts more naturally than a simple fixed time window.

Q23. Two Sum and its variants.

Simple Answer

Given an array and a target, I can use a HashMap to remember values already seen. For each number x , I check whether $\text{target} - x$ is already in the map.

Practical Example

For [2, 7, 11, 15] and target 9, when I reach 7 I look for 2, which I have already seen. The answer is indices 0 and 1. This gives average $O(n)$ time.

Interview Tip

Know the hash-map solution and be ready for variations such as duplicates or returning values instead of indices.

Q24. Valid parentheses, and bracket matching in general.

Simple Answer

Use a stack. Push opening brackets. When a closing bracket appears, check that it matches the most recent opening bracket. At the end, the stack must be empty.

Practical Example

For "[{}]" the stack becomes {, [, (, then each closing bracket matches the correct opening bracket. For "([)]", the order is wrong, so the string is invalid.

Interview Tip

This is a classic example of LIFO behaviour and often extends naturally to JSON-like structures.

Q25. Given two JSON API responses, write a deep diff that reports every differing path.

Simple Answer

A deep diff recursively compares objects, arrays and primitive values. It should report missing keys, additional keys, type differences and different values using a path such as user.address.city.

Practical Example

If response A has user.name='Swapnil' and response B has user.name='Rahul', the result can report path: user.name, expected: Swapnil, actual: Rahul.

Interview Tip

This is especially useful for API regression testing because it tells you exactly where two complex responses differ.

Q26. The endpoint returns 200. What else do you assert before calling it tested?

Simple Answer

I verify response schema and data types, business rules, headers, important response values, response time, database or event side effects, and negative scenarios. HTTP 200 alone is not enough.

Practical Example

A POST /claim may return 200 but create the wrong claim status. I would verify the response body, persisted state and downstream event, not only the status code.

Interview Tip

A useful interview line: "200 is where API validation starts, not where it ends."

Q27. What is idempotency, which methods should have it, and how do you test it?

Simple Answer

An operation is idempotent when repeating the same request produces the same intended final state. GET, PUT and DELETE are generally designed to be idempotent; POST usually is not unless the API implements an idempotency mechanism.

Practical Example

For a payment API using an idempotency key, send the same payment request twice with the same key. The system should create one payment rather than charging the customer twice.

Interview Tip

Test both the response and the final system state.

Q28. How do you test authorization, not just authentication?

Simple Answer

Authentication asks "Who are you?" Authorization asks "What are you allowed to do?" I test valid users with different roles, expired/missing tokens, access to another user's resource and privilege boundaries.

Practical Example

User A requests /users/B/profile while authenticated as A. The API should reject the request if A is not allowed to access B's data. This checks horizontal access control.

Interview Tip

A valid token does not automatically mean the user has permission for every resource.

Q29. Contract testing versus end-to-end testing. When is each the right tool?

Simple Answer

Contract tests verify that two services agree on the expected request and response structure. End-to-end tests verify a complete business journey across multiple systems.

Practical Example

For an Order service calling Payment service, a contract test can quickly verify the payment API schema. A smaller number of E2E tests can verify the complete order-to-payment customer journey.

Interview Tip

They solve different problems and should complement each other.

Q30. What is in your negative-testing checklist for a POST endpoint?

Simple Answer

I test missing fields, wrong data types, null values, empty strings, boundary sizes, duplicate requests, malformed JSON, wrong content type, oversized payloads, invalid IDs and security-related input.

Practical Example

For POST /users, I would test missing email, invalid email format, duplicate email, extremely long name, wrong JSON type for age and malformed JSON.

Interview Tip

A systematic negative checklist is more reliable than inventing negative cases during execution.

Q31. How do you test rate limiting?

Simple Answer

I identify the configured limit, send requests up to the boundary, verify the response when the limit is exceeded, check Retry-After or related headers, and verify that access recovers after the window. I also check whether limits are per user, API key or globally applied.

Practical Example

If the limit is 100 requests per minute, request 100 successfully, request 101 and verify the expected 429 response, then wait for the window and confirm requests work again.

Interview Tip

Test the exact boundary, not just “send lots of requests.”

Q32. The API adds new response fields every sprint. How do your tests survive?

Simple Answer

I assert fields that my feature owns strictly while allowing harmless additive fields. I should fail when an important field is removed or its type changes, but not fail simply because another team adds a new optional field.

Practical Example

If an API adds middleName while my test only needs id and email, the test should continue to pass. If email changes from string to object unexpectedly, the test should fail.

Interview Tip

The goal is strictness where it matters and tolerance where change is backward compatible.

Q33. Design the quality gates for a CI/CD pipeline. What runs pre-merge, post-merge, pre-production?

Simple Answer

Pre-merge should contain fast, deterministic unit/API and critical checks. Post-merge can run broader integration and regression tests. Pre-production should run deployment smoke tests and important end-to-end checks. Production rollout should also rely on monitoring and rollback signals.

Practical Example

A pull request might run unit tests, linting and targeted API tests. After merge, the full API regression can run in parallel. Before production, run critical smoke and deployment validation.

Interview Tip

Every gate must be trustworthy; a flaky gate eventually gets ignored.

Q34. Deployment passed every test and still broke production. Run the post-mortem.

Simple Answer

I would investigate without blame: compare production and test configuration, data, traffic volume, feature flags, dependencies and monitoring. Then I would add a preventive test, environment check or monitoring rule for the failure class.

Practical Example

All tests may pass with 100 users, but production receives 100,000 concurrent requests. The defect may be capacity-related rather than a functional test failure.

Interview Tip

A good post-mortem ends with a concrete engineering change, not only a document.

Q35. Pipeline time keeps growing as the suite grows. How do you keep it under fifteen minutes?

Simple Answer

Use parallel execution, change-based test selection, caching, test sharding, faster test layers and removal of redundant or low-value tests. Slow UI tests should not be the default solution for every check.

Practical Example

If a code change affects only the claims API, targeted API tests can run immediately while the broader suite runs separately. Independent tests can also execute across multiple CI agents.

Interview Tip

Treat CI feedback time as an engineering product requirement.

Q36. Canary versus blue-green deployment. How does your test strategy differ between them?

Simple Answer

In canary deployment, I validate the new version on a small percentage of traffic and closely monitor errors, latency and business metrics before increasing traffic. In blue-green, I validate the idle environment before switching traffic from the old environment.

Practical Example

For canary, I might route 5% of traffic to version 2 and watch error rate and latency. For blue-green, I deploy version 2 to the green environment, run smoke checks, then switch traffic when validation is complete.

Interview Tip

Testing includes both automated checks and the monitoring signals used for rollback.

Q37. Where do your test environments get their data?

Simple Answer

I prefer synthetic or seeded test data for repeatability and safety. If production-like data shape is needed, anonymised samples can help. Raw customer data should not be copied into test environments.

Practical Example

For a healthcare application, I can generate synthetic member records with realistic formats instead of copying real patient information.

Interview Tip

The key trade-off is realism versus privacy and repeatability.

Q38. How do you test infrastructure-as-code changes before they hit production?

Simple Answer

Treat infrastructure code like application code: review the diff, run syntax and policy checks, generate a plan, deploy to a safe staging account, validate the result and have a tested rollback path.

Practical Example

For a Terraform change that creates an AWS resource, I review terraform plan, run policy checks, apply it in a non-production account and verify the resource and permissions before production deployment.

Interview Tip

Infrastructure can break production even when application tests are completely green.

Q39. Design a test automation strategy for a brand-new product, from zero.

Simple Answer

I start with business risk and critical user journeys, not with a tool. Then I define test layers—unit, API/integration, UI and performance—followed by test data, environments, CI, reporting, ownership and rollout.

Practical Example

For a new insurance application, I identify critical journeys such as registration, eligibility, plan selection and payment. I automate business rules at lower levels, add API coverage, and keep UI automation focused on critical journeys.

Interview Tip

Tools come after the strategy. A good design should survive a tool change.

Q40. Design a real-device testing lab, or a cross-browser grid.

Simple Answer

I need device/browser allocation, a queue, isolation between sessions, cleanup, health monitoring, logs, screenshots/video and protection against two jobs claiming the same device.

Practical Example

A mobile test requests a Pixel device. The grid assigns it to one worker, resets the application state, runs the test, collects logs/video and releases the device. If the device becomes unhealthy, the grid marks it unavailable.

Interview Tip

Think of the grid as an infrastructure system, not just a collection of browsers.

Q41. Design the reporting system for ten thousand test runs a day.

Simple Answer

I would store results centrally, deduplicate failures, retain history, detect recurring flakes, provide dashboards by team and alert on new failures separately from known failures.

Practical Example

If a test has failed 400 times with the same known issue, the dashboard should distinguish it from a brand-new failure affecting production validation.

Interview Tip

History-aware reporting gives context instead of treating every red test as a new incident.

Q42. The product integrates with a thousand external services. Design the testing approach.

Simple Answer

I would not create full E2E tests for every integration. I would use contract tests, provider sandboxes or recorded responses, continuous verification, a thin layer of critical E2E flows and monitoring for provider changes.

Practical Example

For 1,000 payment or identity providers, contract tests can validate each integration cheaply. A few E2E tests verify the most important end-to-end customer journeys.

Interview Tip

The objective is broad confidence without creating an impossible maintenance burden.

Q43. Design load testing for a live event where a million users arrive in two minutes.

Simple Answer

This is a sudden-spike or thundering-herd scenario. I would model a realistic ramp, test queues and backpressure, measure autoscaling reaction time, pre-scale capacity when necessary and monitor latency/error rates.

Practical Example

If one million users arrive in two minutes but autoscaling needs ten minutes to add capacity, relying only on autoscaling is insufficient. I would pre-scale resources and test the spike before the event.

Interview Tip

Load-test the traffic shape that will actually happen, not only a gentle linear ramp.

Q44. How do you test a zero-downtime migration?

Simple Answer

I would validate data before, during and after migration using reconciliation, checksums or business-level comparisons. I would use shadow reads or dual writes where appropriate, canary the cutover and test rollback before migration day.

Practical Example

During a database migration, old and new systems can temporarily receive the same writes. I compare record counts and important field values, verify application behaviour against the new system, and keep a tested rollback path.

Interview Tip

Matching row counts alone is not enough; the content and behaviour must also match.

Q45. The feature's output is generated by a model and different every run. How do you test it?

Simple Answer

I would not rely only on exact string matching. I would test properties such as required information, format, safety constraints and correctness against a defined rubric. I would evaluate many runs and measure a success rate.

Practical Example

For an AI assistant that creates test cases, I can check that every output contains a valid test title, steps and expected result, and that required business rules are covered. Human-reviewed evaluation examples can measure quality across many runs.

Interview Tip

For probabilistic output, test the properties and quality of the result rather than demanding one exact sentence.

Q46. What is LLM-as-judge, and what are its failure modes?

Simple Answer

LLM-as-judge means using one model to evaluate another model's output against a rubric or reference. It can be useful, but the judge itself can be inconsistent or biased.

Practical Example

If an AI model generates an API test case, a second model can score it for correctness, completeness and relevance. I would periodically compare those scores with human-labelled examples.

Interview Tip

Use a fixed rubric, monitor judge agreement and perform human spot checks.

Q47. How do you build an evaluation set for an AI feature?

Simple Answer

I collect representative real-world cases, difficult cases and edge cases, label expected behaviour where possible, and continuously add important production failures to the evaluation set.

Practical Example

For an AI claims assistant, the evaluation set can include normal claims, missing information, ambiguous requests, unusual wording and known production failures. Every serious new failure becomes a regression evaluation case.

Interview Tip

The evaluation set should grow as the product learns what can go wrong.

Q48. An AI agent picks the right tool 95 percent of the time. How would you catch a silent regression to 90?

Simple Answer

Run a fixed evaluation set on every meaningful model or agent change, compare the new score with a baseline, and define a regression threshold. Do not depend only on exceptions because the wrong tool can be selected successfully from a technical perspective.

Practical Example

If an agent previously selected the correct tool for 950 of 1,000 evaluation cases and a new version selects correctly only 900, the evaluation dashboard should detect the drop and block or flag the change according to the agreed threshold.

Interview Tip

Measure behaviour, not just whether the program threw an error.

Q49. How do you use AI tools in your own testing work?

Simple Answer

I use AI for tasks such as test scaffolding, exploring unfamiliar code, generating initial test ideas, explaining errors and improving repetitive work. I review and verify every generated change before it enters the codebase.

Practical Example

For example, I can use Cursor or Claude with repository rules and skills to help create Playwright test scaffolding. I still verify selectors, assertions, framework conventions, security and actual application behaviour before committing the code.

Interview Tip

Show ownership: AI can accelerate engineering work, but the engineer remains responsible for the result.

Q50. Will AI replace testers?

Simple Answer

AI can automate parts of test generation, execution and analysis, but testing still requires defining risk, deciding what should be verified, interpreting failures and evaluating systems whose behaviour may be probabilistic.

Practical Example

A traditional UI check may be generated by AI, but someone still needs to decide whether the right business risks are covered. For an AI feature, the tester also needs to define evaluation criteria and monitor quality across many outputs.

Interview Tip

A balanced interview answer is: AI changes the tester's work and increases the importance of engineering, evaluation and quality judgement; it does not make quality responsibility disappear.